

32-bit MIPS
Pipelined
FPGA Optimization

Amir Jamshidi Bandari
Electrical Engineering
Toronto Metropolitan University
May 2026

OPTIMIZATION SUMMARY:

Multiplier:

Summary	Before Optimization	After Optimization
WNS (ns)	0.402	0.132
WHS (ns)	0.126	0.105
Frequency (MHz)	60	74.906
Period (ns)	16.670	13.350

Pipeline Execution Stage:

Summary	Before Optimization	After Optimization
WNS (ns)	0.132	0.01
WHS (ns)	0.105	0.116
Frequency (MHz)	74.906	96.618
Period (ns)	13.350	10.350

Divider:

Summary	Before Optimization	After Optimization
WNS (ns)	0.01	0.092
WHS (ns)	0.116	0.113
Frequency (MHz)	96.618	94.340
Period (ns)	10.350	10.600

HOW ARE HARDWARE VERIFIED AND/OR OPTIMIZED:

- Verification

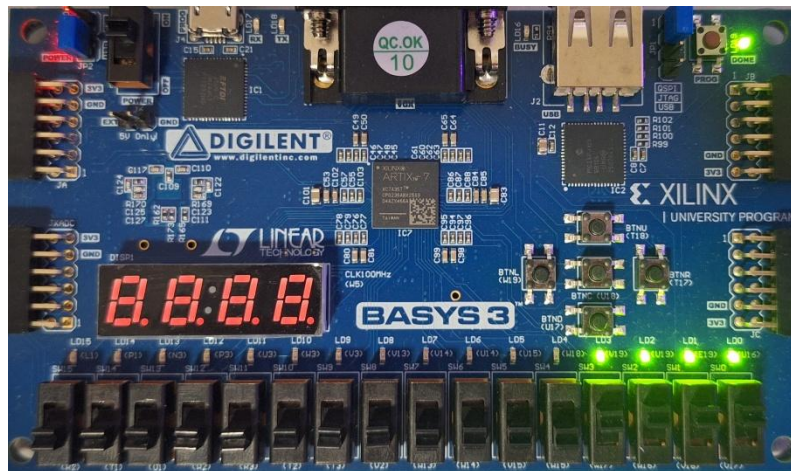
For verification, we will focus on the switches and LEDs on the Basys 3 FPGA board, the design is checked by comparing register values with expected values, if the results match, the assigned LEDs will turn on, if the results don't match, the LEDs will stay off.

For example: `assign led[1] = (Reg1 == 32'd5) && switch[1];`

This means if register 1 has an expected value of 5 and if switch 1 is on, turn the LED 1 on.

In situations with off LED, debugging is done by checking register values, by assigning each bit of the register to each LED, an ON LED is a value of 1 and an OFF LED is a value of 0.

For example:



The picture shows the lower 16 bits of a register, the lower 16 bits have a value of 0000000000001111 in binary, which is a value of 15. These values give hints for debugging such as if wrong or no value is forwarded, if new value didn't replace old value. Other ways are also used to help debug hardware, such as isolating the instruction outputting wrong value, creating temporary registers at each stage and connecting them to LEDs to find which stage produces wrong value. Since number of LEDs are limited, switches can be used as toggles to check different registers in a single test.

- Optimization

The FPGA implementation of this project focuses on optimization by identifying the critical path of the design. Critical path shows the slowest path that limits clock cycle time, and this limitation determines the minimum clock period for the entire pipeline.

Critical path is identified by the lowest Worst Negative Slack (WNS), which represents the path with the smallest timing margin, meaning the least available time for data to arrive and be ready before the next clock edge. If data is not ready before the next clock edge, a setup time violation occurs, which can propagate incorrect values through the pipeline.

Ways to optimize the design are introduce pipeline registers to break long combinational paths, use multicycle execution for high latency units such as multipliers or dividers, reduce unnecessary fanout and simplify datapath logic.

EXAMPLE INSTRUCTIONS USED FOR TESTING:

Below are example instructions tested, verified, and used to find the critical paths.

1. Register Initialization

```
addi $1,$0,5    // R1 = 5
addi $2,$0,7    // R2 = 7
addi $3,$0,9    // R3 = 9
```

Initializes base register values used throughout the program.

2. Immediate Construction (LUI / ORI)

```
lui $4,0x1234
ori $4,$4,0x5678 // R4 = 0x12345678
```

Builds a 32-bit constant using upper and lower immediate instructions.

3. ALU Operations and Forwarding

```
add $5,$1,$2    // R5 = 12
beq $5,$2, +2    // not taken
add $8,$5,$3     // R8 = 21 (tests forwarding)
add $8,$8,$2     // R8 = 28 (chain dependency)
add $6,$5,$3     // R6 = 21
add $7,$6,$2     // R7 = 28
```

Tests ALU functionality, forwarding paths, and branch-not-taken behavior.

4. Load/Store and Load-Use Hazard

```
add $9,$7,$1    // R9 = 33
sw $9,0($0)
lw $10,0($0)
add $10,$10,$2  // R10 = 40 (load-use hazard)
```

Verifies memory operations and handling of load-use hazards.

5. Multiply (Small Values)

```
mult $10,$10
mflo $11        // R11 = 1600
mfhi $12        // R12 = 0
```

Tests multiplication and correct usage of HI/LO registers.

6. Memory Dependency

```
sw $11,4($3)
sw $12,8($3)
lw $13,4($3) // R13 = 1600
lw $14,8($3) // R14 = 0
```

Checks correct memory write and read behavior with dependent addresses.

7. ALU Dependency Chain

```
add $14,$13,$2 // R14 = 1607
add $15,$14,$3 // R15 = 1616
```

Tests chained dependencies and forwarding across multiple stages.

8. Control Hazard (Branch Taken + Flush Verification)

```
beq $15,$15, +2 // always taken
addi $9,$0,1    // SHOULD BE FLUSHED
addi $9,$0,2    // SHOULD BE FLUSHED
add $9,$9,$2    // R9 = 40 (correct if flush works)
```

Verifies control hazard handling.

If flushing fails, incorrect instructions execute and:

R9 = 9 (incorrect result)

9. Multiply (Large Values)

```
mult $4,$4
mflo $16 // R16 = 0x1DF4D840
mfhi $17 // R17 = 0x014B66DC
```

Tests large multiplication and correctness of HI/LO outputs.

10. Final Result

```
add $18,$16,$0 // R18 = 0x1DF4D840
```

Final verification of correct data propagation and writeback.

11.

```
0142001A // div $10,$2 (example: 40 / 7 = 5 remainder 5)
00005812 // mflo $11 → quotient
00006010 // mfhi $12 → remainder
```

Divide 40 by 7, store the result and remainder inside register 11 and register 12.

MULTIPLIER OPTIMIZATION:

Original Critical Path:

In this test, the critical path was found in the execute stage. The path was:

ID Pipeline Register → Multiplier Input MUXes → Multiplier → HI/LO MUXes → EX Pipeline Register

Before optimization, the design had the following timing results:

Timing		Timing	
Worst Negative Slack (WNS):	0.402 ns	Worst Hold Slack (WHS):	0.126 ns
Total Negative Slack (TNS):	0 ns	Total Hold Slack (THS):	0 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	3434	Total Number of Endpoints:	3434

With the current design, the CPU could run at about **60 MHz**.

Clock Summary			
Name	Waveform	Period (ns)	Frequency (MHz)
sys_clk_pin	{0.000 5.000}	16.670	59.988

Root Cause Analysis:

The multiplier is a logic-intensive unit that uses multiple DSP blocks and LUTs. In the original design, multiplication was performed in a single cycle. This extended the critical path and forced the CPU to operate at a lower clock frequency.

The timing report shows that the critical path starts at the ID pipeline register and ends at the EX pipeline register. The lowest-slack path is path 1, meaning it is the most timing-critical path in the design. Although the top ten paths show similar “From” and “To” points, they are all related to multiplier operation.

This is confirmed by comparing the logic delay and net delay. The logic delay dominates the net delay, meaning the multiplication operation itself contributes more delay than routing distance.

Intra-Clock Paths - sys_clk_pin - Setup										
Name	Slack ¹	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	
Path 1	0.402	20	9	140	id_pipe/rt_EX_reg[0]/C	ex_pipe/HI_send_ME_reg[29]/D	16.230	9.041	7.189	
Path 2	0.425	22	9	140	id_pipe/rt_EX_reg[0]/C	ex_pipe/HI_send_ME_reg[31]/D	16.205	9.252	6.953	
Path 3	0.429	19	9	140	id_pipe/rt_EX_reg[0]/C	ex_pipe/HI_send_ME_reg[25]/D	16.252	8.924	7.328	
Path 4	0.476	20	9	140	id_pipe/rt_EX_reg[0]/C	ex_pipe/HI_send_ME_reg[30]/D	16.155	8.952	7.203	
Path 5	0.603	21	9	140	id_pipe/rt_EX_reg[0]/C	ex_pipe/HI_send_ME_reg[27]/D	16.076	9.138	6.938	
Path 6	0.648	19	9	140	id_pipe/rt_EX_reg[0]/C	ex_pipe/HI_send_ME_reg[26]/D	16.032	8.835	7.197	
Path 7	0.656	18	9	140	id_pipe/rt_EX_reg[0]/C	ex_pipe/HI_send_ME_reg[23]/D	15.975	8.800	7.175	
Path 8	0.656	20	9	140	id_pipe/rt_EX_reg[0]/C	ex_pipe/HI_send_ME_reg[28]/D	15.972	8.926	7.046	
Path 9	0.663	21	9	140	id_pipe/rt_EX_reg[0]/C	ex_pipe/HI_send_ME_reg[24]/D	16.019	9.040	6.979	
Path 10	0.677	19	9	140	id_pipe/rt_EX_reg[0]/C	ex_pipe/HI_send_ME_reg[19]/D	16.002	8.910	7.092	

Optimization Strategy:

The multiplier was changed into a multicycle unit. For 32-bit operands, multiplication can be performed over multiple cycles, up to 32 cycles, instead of completing in one clock cycle. This significantly reduces combinational delay at the cost of increased latency.

Before optimization:

The multiplier receives two operands, multiplies them in a single cycle, stores the result as a 64-bit value, and outputs the upper and lower 32-bit values.

```

always_comb begin
    mult_result = 64'b0;
    mult_resultu = 64'b0;
    case (ALU_Control_EX)
        ALU_MULT: begin
            mult_result = $signed(data_out_1_EX_mux) * $signed(data_out_2_EX_mux);
        end
    endcase
end

assign mult_HI_EX = (ALU_Control_EX == ALU_MULT) ? mult_result[63:32] : 32'b0;
assign mult_LO_EX = (ALU_Control_EX == ALU_MULT) ? mult_result[31:0] : 32'b0;

```

After Optimization:

The multicycle multiplier is separated into control logic, execution logic, and output logic.

For the control logic, the multiplier first receives the ALU_CONTROL signal to begin execution. When this signal is received, the internal MULTIPLY signal is set high, indicating that multiplication is currently in progress. This signal is necessary for stalling dependent instructions. Stall detection occurs in the decode and execute stages, and the pipeline remains stalled until the multiplication process is complete.

After multiplication is complete, the multiplier outputs the HI/LO control signals needed to store the final result in the HI/LO registers. These HI/LO signals no longer come directly from the main control unit because their timing must match the completion of the multicycle multiplier.

For the execution logic, multiplication only occurs while the MULTIPLY signal is high. During this process, addition and bit shifting are used to build the 64-bit result. A cycle counter tracks the number of cycles and stops the process after 32 cycles.

For the output logic, the 64-bit result is split into two 32-bit values. The upper 32 bits are sent to HI, and the lower 32 bits are sent to LO.

To make the code easier to understand, explanatory comments are included using `//`.

```
// Control logic of the multiplier.
always_comb begin
    // Default values.
    Multiply = 0;
    stall_mult = 0;
    Multicycle_HILO_Register_Enable_EX = 0;
    Multicycle_HILO_Select_EX = HILO_NONE;

    // When control unit signal is received, begin multiplier multicycle execution.
    // Multicycle execution must be 31 clock cycles long due to 32-bit values.
    if ((ALU_Control_EX == ALU_MULT) || (cycle > 0 && cycle < 6'd32))
        // While these cycles are being counted, signal is sent to multiplier for continuous execution.
        Multiply = 1;

    // If any dependent instruction is detected, the pipeline must be stalled for the
    // remaining cycles, to finish multiplication.
    if (((Writeback_Control_ID == WB_HI) || (Writeback_Control_ID == WB_LO)) && (cycle < 6'd32) && Multiply) begin
        stall_mult = 1;
    end

    // During 32th cycle, multiplication is finished, signals must be send to HI/LO register and their multiplexers
    // to store the results.
    if (cycle == 6'd32) begin
        Multiply = 0;
        Multicycle_HILO_Register_Enable_EX = 1;
        Multicycle_HILO_Select_EX = HILO_MULT;
    end
end
end
```

```

// Execution logic of the multiplier.
always_ff @(posedge clk or posedge reset) begin
    // Default values during reset.
    if (reset) begin
        mult_result <= 64'b0;
        cycle <= 0;
        mult_operand_1 <= 0;
        mult_operand_2 <= 0;
    end

    // When control logic sends signal, begin multiplication.
    else if (Multiply) begin

        // At cycle 0, pipeline inputs are copied to safe multiplier registers, and the 32 cycles begin.
        if (cycle == 0) begin
            mult_result <= 64'b0;
            mult_operand_1 <= data_out_1_EX_mux;
            mult_operand_2 <= data_out_2_EX_mux;
            cycle <= 1;
        end

        // The multiplication process, which includes shifting both operand bits and adding the operand with its previous cycle result
        // only if second operand has the lowest bit equal to 1 (HIGH).
        else begin
            if (mult_operand_2[0]) begin
                mult_result <= mult_result + {{32{mult_operand_1[31]}}, mult_operand_1};
            end
            mult_operand_1 <= mult_operand_1 << 1;
            mult_operand_2 <= mult_operand_2 >> 1;
            cycle <= cycle + 1;
        end
    end

    // When control logic doesn't send any multiplication signal, set cycle to its default value.
    else begin
        cycle <= 0;
    end
end
end

```

```

// Output logic of multiplier.
assign mult_HI_EX = (cycle == 6'd32) ? mult_result[63:32] : 32'b0;
assign mult_LO_EX = (cycle == 6'd32) ? mult_result[31:0] : 32'b0;

```

FPGA Timing Results:

Setup	Hold
Worst Negative Slack (WNS): 0.132 ns	Worst Hold Slack (WHS): 0.105 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 3759	Total Number of Endpoints: 3759
All user specified timing constraints are met.	

Clock Summary			
Name	Waveform	Period (ns)	Frequency (MHz)
sys_clk_pin	{0.000 5.000}	13.350	74.906

After implementing the multicycle multiplier, timing closure improved. Through several timing tests, the lowest stable period was found to be **13.350 ns**. Any lower period failed timing closure.

The final frequency increased from approximately **60 MHz** to **74.906 MHz**, while still maintaining positive WNS.

Tradeoffs:

Multicycling the multiplier reduces the amount of combinational logic used in a single cycle. This decreases the timing pressure caused by DSP blocks and LUTs in the original single-cycle multiplier.

The main tradeoff is latency. The new multiplier takes up to **32 cycles** to complete one multiplication operation. This means that a dependent instruction, such as mflo or mfhi, may stall the CPU until the multiplication result is ready.

Another tradeoff is increased control complexity. The multiplier now requires additional signals for stalling, HI/LO register control, result selection, cycle counting, and internal multicycle execution.

However, this tradeoff is worthwhile because the CPU frequency increased significantly.

Conclusion:

The multiplier optimization was successful. By sacrificing multiplication latency, the design achieved higher overall throughput and improved timing closure. The CPU frequency increased by about **15 MHz**.

After optimization, the multiplier no longer dominated the CPU critical path as a single-cycle logic-intensive unit.

EXECUTION STAGE OPTIMIZATION:

Original Critical Path:

Name	Slack ^{^1}	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
↳ Path 1	0.132	20	13	140	ex_pipe/data_in_address_ME_reg[3]/C	if_pipe/instruction_ID_reg[22]/CE	12.946	4.122	8.824
↳ Path 2	0.132	20	13	140	ex_pipe/data_in_address_ME_reg[3]/C	if_pipe/instruction_ID_reg[22]_rep/CE	12.946	4.122	8.824
↳ Path 3	0.132	20	13	140	ex_pipe/data_in_address_ME_reg[3]/C	if_pipe/instruction_ID_reg[2]/CE	12.946	4.122	8.824
↳ Path 4	0.132	20	13	140	ex_pipe/data_in_address_ME_reg[3]/C	if_pipe/instruction_ID_reg[3]/CE	12.946	4.122	8.824
↳ Path 5	0.137	20	13	140	ex_pipe/data_in_address_ME_reg[3]/C	id_pipe/data_out_1_EX_reg[19]/CE	12.944	4.122	8.822
↳ Path 6	0.137	20	13	140	ex_pipe/data_in_address_ME_reg[3]/C	id_pipe/data_out_1_EX_reg[4]/CE	12.944	4.122	8.822
↳ Path 7	0.137	20	13	140	ex_pipe/data_in_address_ME_reg[3]/C	id_pipe/data_out_2_ALUSrc_EX_reg[12]/CE	12.944	4.122	8.822
↳ Path 8	0.137	20	13	140	ex_pipe/data_in_address_ME_reg[3]/C	id_pipe/data_out_2_ALUSrc_EX_reg[13]/CE	12.944	4.122	8.822
↳ Path 9	0.137	20	13	140	ex_pipe/data_in_address_ME_reg[3]/C	id_pipe/data_out_2_ALUSrc_EX_reg[4]/CE	12.944	4.122	8.822
↳ Path 10	0.137	20	13	140	ex_pipe/data_in_address_ME_reg[3]/C	id_pipe/data_out_2_ALUSrc_EX_reg[5]/CE	12.944	4.122	8.822

After the multiplier optimization, the new critical path was found across the execute stage. In the timing report, the path begins at the **EX pipeline register** and ends at the **ID/IF pipeline register**.

The timing period was kept the same as the previous multiplier-optimized design. However, the critical path was no longer dominated by a single hardware unit such as the multiplier. Instead, the delay was spread across the complete execute-stage datapath.

The net delay is more than double the logic delay. This shows that the main issue is no longer the logic operation itself. Instead, the delay is mainly caused by routing distance and long combinational paths across the execute stage.

This suggests that the execute stage contains too much hardware in one cycle, causing signals to travel through multiple units and long routes before reaching the next pipeline register.

Root Cause Analysis:

Delay Type	Incr (ns)	Path (ns)	Location	Cell Pin	Cell	Netlist Resources
<u>FDCE (Prop fdce C Q)</u>	(f) 0.419	5.508	Site: SLICE_X13Y7	Q	data_in_address_ME_reg[3]	ex_pipe/data_in_address_ME_reg[3]/Q
net (fo=5, routed)	0.616	6.125				ex_pipe/data_in_address_ME_reg[4]_0[3]
<u>LUT4 (Prop lut4 I0 O)</u>	(r) 0.297	6.422	Site: SLICE_X12Y9	O	mult_operand_2[31]_i_27	ex_pipe/mult_operand_2[31]_i_27/O
net (fo=1, routed)	0.791	7.213				id_pipe/mult_operand_2[31]_i_10
<u>LUT6 (Prop lut6 I4 O)</u>	(r) 0.124	7.337	Site: SLICE_X15Y9	O	mult_operand_2[31]_i_18	id_pipe/mult_operand_2[31]_i_18/O
net (fo=5, routed)	0.754	8.091				ex_pipe/mult_operand_2_reg[31]_3
<u>LUT6 (Prop lut6 I2 O)</u>	(r) 0.124	8.215	Site: SLICE_X13Y8	O	mult_operand_2[31]_i_13	ex_pipe/mult_operand_2[31]_i_13/O
net (fo=140, routed)	1.016	9.231				ex_pipe/Writeback_Control_WB_reg[0][1]
<u>LUT6 (Prop lut6 I2 O)</u>	(r) 0.124	9.355	Site: SLICE_X12Y2	O	ALU_result_ME[5]_i_10	ex_pipe/ALU_result_ME[5]_i_10/O
net (fo=2, routed)	0.699	10.054				ex_pipe/ALU_result_ME[5]_i_10_n_0
<u>LUT6 (Prop lut6 I5 O)</u>	(r) 0.124	10.178	Site: SLICE_X10Y2	O	ALU_result_ME[5]_i_9	ex_pipe/ALU_result_ME[5]_i_9/O
net (fo=4, routed)	0.419	10.597				id_pipe/ALU_result_ME_reg[5]
<u>LUT3 (Prop lut3 I1 O)</u>	(r) 0.124	10.721	Site: SLICE_X11Y4	O	pc_adder_value_ID[6]_i_9	id_pipe/pc_adder_value_ID[6]_i_9/O
net (fo=3, routed)	0.607	11.328				ex_pipe/ALU_result_ME_reg[27]_i_10_0[1]
<u>LUT2 (Prop lut2 I1 O)</u>	(r) 0.124	11.452	Site: SLICE_X10Y6	O	ALU_result_ME[7]_i_15	ex_pipe/ALU_result_ME[7]_i_15/O
net (fo=1, routed)	0.000	11.452				ex_pipe/ALU_result_ME[7]_i_15_n_0
<u>CARRY4 (Prop c...4 S[1] CO[3])</u>	(r) 0.533	11.985	Site: SLICE_X10Y6	CO[3]	ALU_result_ME_reg[7]_i_6	ex_pipe/ALU_result_ME_reg[7]_i_6/CO[3]
net (fo=1, routed)	0.000	11.985				ex_pipe/ALU_result_ME_reg[7]_i_6_n_0
<u>CARRY4 (Prop carry4 CI CO[3])</u>	(r) 0.117	12.102	Site: SLICE_X10Y7	CO[3]	ALU_result_ME_reg[11]_i_1	ex_pipe/ALU_result_ME_reg[11]_i_1/CO[3]
net (fo=1, routed)	0.000	12.102				ex_pipe/ALU_result_ME_reg[11]_i_1_n_0
<u>CARRY4 (Prop carry4 CI CO[3])</u>	(r) 0.117	12.219	Site: SLICE_X10Y8	CO[3]	ALU_result_ME_reg[15]_i_1	ex_pipe/ALU_result_ME_reg[15]_i_1/CO[3]
net (fo=1, routed)	0.000	12.219				ex_pipe/ALU_result_ME_reg[15]_i_1_n_0
<u>CARRY4 (Prop carry4 CI CO[3])</u>	(r) 0.117	12.336	Site: SLICE_X10Y9	CO[3]	ALU_result_ME_reg[19]_i_1	ex_pipe/ALU_result_ME_reg[19]_i_15/CO[3]
net (fo=1, routed)	0.000	12.336				ex_pipe/ALU_result_ME_reg[19]_i_15_n_0
<u>CARRY4 (Prop carry4 CI CO[3])</u>	(r) 0.117	12.453	Site: SLICE_X10Y10	CO[3]	ALU_result_ME_reg[23]_i_1	ex_pipe/ALU_result_ME_reg[23]_i_11/CO[3]
net (fo=1, routed)	0.000	12.453				ex_pipe/ALU_result_ME_reg[23]_i_11_n_0
<u>CARRY4 (Prop carry4 CI CO[3])</u>	(r) 0.117	12.570	Site: SLICE_X10Y11	CO[3]	ALU_result_ME_reg[27]_i_1	ex_pipe/ALU_result_ME_reg[27]_i_10/CO[3]
net (fo=1, routed)	0.000	12.570				ex_pipe/ALU_result_ME_reg[27]_i_10_n_0
<u>CARRY4 (Prop carry4 CI O[1])</u>	(f) 0.323	12.893	Site: SLICE_X10Y12	O[1]	ALU_result_ME_reg[31]_i_1	ex_pipe/ALU_result_ME_reg[31]_i_15/O[1]
net (fo=1, routed)	0.765	13.658				id_pipe/data0[29]
<u>LUT3 (Prop lut3 I1 O)</u>	(f) 0.306	13.964	Site: SLICE_X13Y19	O	ALU_result_ME[29]_i_5	id_pipe/ALU_result_ME[29]_i_5/O
net (fo=1, routed)	0.404	14.368				id_pipe/ALU_result_ME[29]_i_5_n_0
<u>LUT6 (Prop lut6 I0 O)</u>	(f) 0.124	14.492	Site: SLICE_X13Y20	O	ALU_result_ME[29]_i_3	id_pipe/ALU_result_ME[29]_i_3/O
net (fo=1, routed)	0.000	14.492				id_pipe/ALU_result_ME[29]_i_3_n_0
<u>MUXF7 (Prop muxf7 I1 O)</u>	(f) 0.245	14.737	Site: SLICE_X13Y20	O	ALU_result_ME_reg[29]_i_1	id_pipe/ALU_result_ME_reg[29]_i_1/O
net (fo=3, routed)	0.602	15.339				id_pipe/ALU_Control_EX_reg[0]_0[29]
<u>LUT4 (Prop lut4 I2 O)</u>	(r) 0.298	15.637	Site: SLICE_X11Y18	O	instruction_ID[31]_i_11	id_pipe/instruction_ID[31]_i_11/O
net (fo=1, routed)	0.780	16.417				id_pipe/instruction_ID[31]_i_11_n_0
<u>LUT6 (Prop lut6 I3 O)</u>	(r) 0.124	16.541	Site: SLICE_X11Y16	O	instruction_ID[31]_i_5	id_pipe/instruction_ID[31]_i_5/O
net (fo=57, routed)	0.744	17.285				id_pipe/instruction_ID[31]_i_5_n_0
<u>LUT6 (Prop lut6 I4 O)</u>	(r) 0.124	17.409	Site: SLICE_X12Y15	O	instruction_ID[31]_i_1	id_pipe/instruction_ID[31]_i_1/O
net (fo=29, routed)	0.627	18.036				if_pipe/E[0]

The expanded critical path shows that the bottleneck comes from multiple operations happening in the same clock cycle. These include:

- ALU operation
- multiplier input routing
- forwarding decisions
- forwarding data paths
- ALU input multiplexers
- memory input forwarding
- multiplier input forwarding

The design contains five 12:1 multiplexers in the execute stage. These multiplexers are used to select forwarded or non-forwarded values for different execution-stage inputs.

Having all of these operations in one stage increases both logic delay and routing delay.

The main issue is not only the amount of logic, but also the physical routing distance between pipeline registers, forwarding sources, multiplexers, and arithmetic units. Because forwarding data must come from multiple pipeline stages, the routing network becomes large and timing-critical.

Therefore, the execute stage became too large to operate efficiently within one clock cycle.

Optimization Strategy:

To reduce combinational delay and routing distance, the execute stage was split into two separate pipeline stages by inserting a new pipeline register between operand selection and arithmetic execution.

The update is shown below:

Before Optimization: ID_pipe → Multiplexers → ALU / MULT / DIV → EX_pipe.

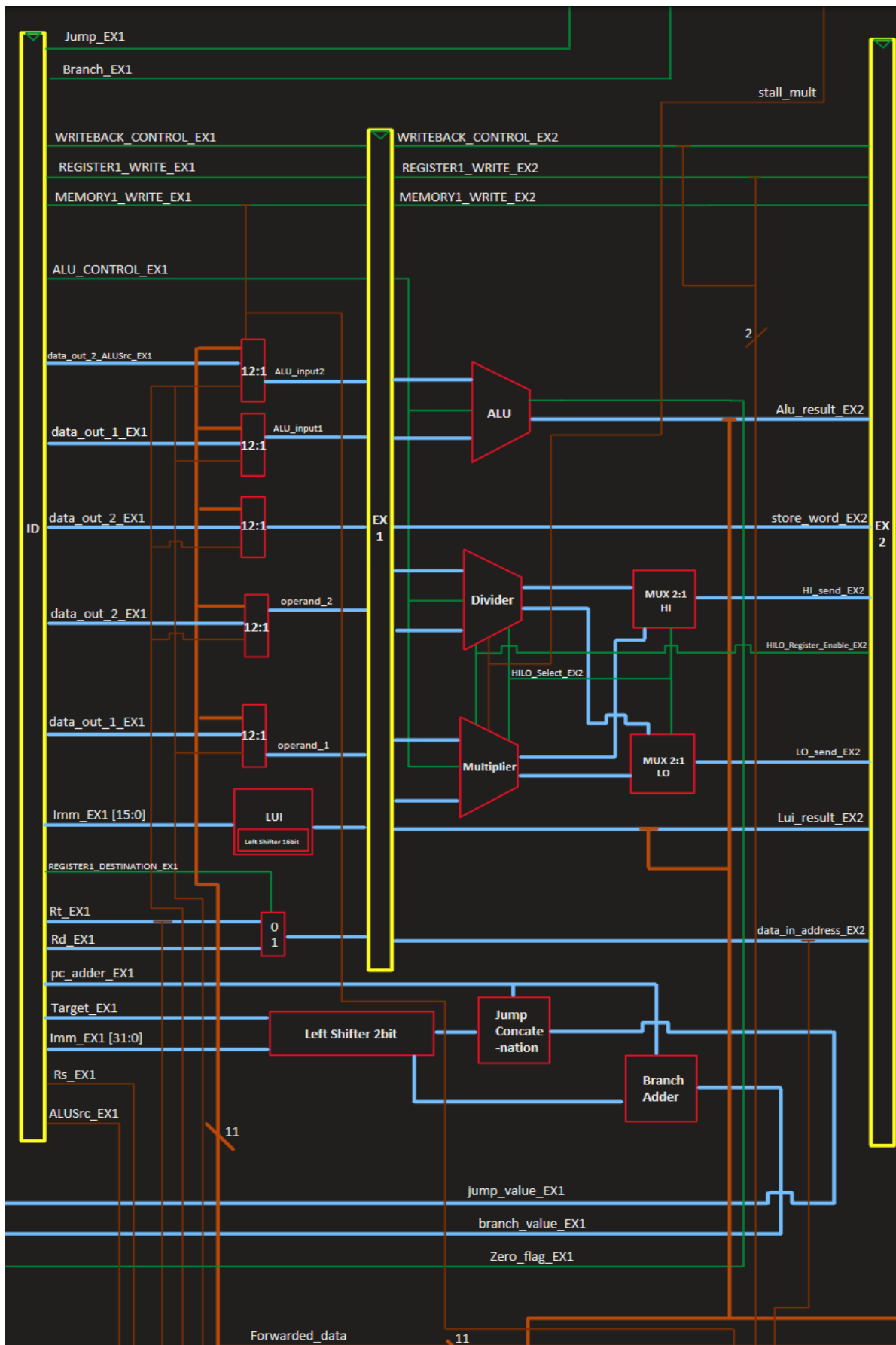
After Optimization: ID_pipe → Multiplexers → EX1_pipe → ALU / MULT / DIV → EX2_pipe.

Inserting the EX1 pipeline register required several major design changes:

- Forwarding now occurs from the EX2, MEM, and WB stages instead of only MEM and WB.
- Stall detection now checks both EX1 and EX2 stages instead of only one execute stage.
- Input multiplexers, bit shifting, destination address selection, branch logic, and jump logic remain in EX1.
- ALU, divider, multiplier, and HI/LO register multiplexers are moved into EX2.

This optimization reduces the amount of combinational logic in each stage and shortens the routing distance between dependent hardware blocks.

Here is the diagram of updated execute stage pipeline:



The updated execute stage is divided into two major parts: the stage between **ID and EX1**, and the stage between **EX1 and EX2**.

In the stage between the ID and EX1 pipeline registers, the dominant hardware blocks are the five 12:1 multiplexers. These multiplexers must remain separate from the arithmetic units. If a single shared multiplexer were used, the routing distance between the pipeline registers and arithmetic units would increase. The duplicated multiplexers are used as an optimization to keep each multiplexer physically closer to its dependent hardware, while sacrificing fanout.

In this stage, forwarding decisions and operand preparation occur. The **dark brown** thin lines represent **forwarding control signals** feeding into the multiplexers OR signals used by **forwarding unit**. There are two main forwarding control signals:

Forward1: forwarding control for the first operand.

Forward2: forwarding control for the second operand.

The thick **orange** lines represent **forwarded data**. These are the 11 possible forwarded inputs feeding the five multiplexers. The thick **light blue** lines represent **non-forwarded datapath** signals. The thin **dark green** lines represent **control unit signals**.

Several other hardware blocks also exist in this stage, pipelined hardware include the LUI and the register destination selection multiplexer. These blocks contribute much less delay compared to the forwarding logic.

The non-pipelined hardware includes branch and jump execution. These are handled through control hazards in the EX1 stage, so their values do not need to propagate further through the pipeline. However, they may still become future bottlenecks because they control flushing of the IF and ID pipeline registers.

In the stage between the EX1 and EX2 pipeline registers, arithmetic operations occur. The dominant hardware blocks in this stage are the multiplier and divider units. Both operate over multiple cycles and eventually output HI and LO results along with the control signals needed to store those results.

Other units in EX2, such as the ALU and HI/LO multiplexers, contribute less delay compared to the multicycle arithmetic units.

FPGA Timing Results:

After implementing the new six-stage pipeline design, the FPGA timing results improved significantly.

The optimized design reached:

Setup	Hold
Worst Negative Slack (WNS): 0.010 ns	Worst Hold Slack (WHS): 0.116 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 3823	Total Number of Endpoints: 3823

All user specified timing constraints are met.

Name	Waveform	Period (ns)	Frequency (MHz)
sys_clk_pin	{0.000 5.000}	10.350	96.618

The clock period decreased from **13.350 ns** to **10.350 ns**, increasing the frequency from approximately **74.906 MHz** to **96.618 MHz**.

The optimization reduced combinational delay by separating forwarding and operand selection from arithmetic execution. It also reduced routing pressure by shortening the distance between dependent hardware blocks.

Before inserting the new pipeline register, the highest fanout was **140** with approximately **1.016 ns** of delay. After the pipeline split, the highest fanout was **139** with approximately **0.794 ns** of delay. Although fanout did not significantly decrease, the routing delay improved due to better stage separation and shorter local connections.

Tradeoffs:

The main tradeoff of this optimization is increased pipeline complexity.

Changing from a five-stage pipeline to a six-stage pipeline requires more advanced hazard handling. Forwarding and stall logic must now account for an additional execute stage. This increases control complexity, logic usage, and routing complexity.

However, the performance improvement is significant. The added pipeline register reduces the critical path enough to justify the extra design complexity.

In this case, the optimization was necessary because the execute stage had become too large and routing-dominated after the multiplier optimization.

Conclusion:

Inserting a pipeline register into the execute stage successfully improved performance by reducing the amount of combinational logic in a single cycle.

The CPU frequency increased, giving a performance improvement of about **21.712 MHz**.

Separating forwarding and operand selection from arithmetic execution reduced routing distance and improved timing closure. After this optimization, the execute stage was no longer dominated by long routing paths between forwarding logic and arithmetic units.

DIVIDER OPTIMIZATION:

Original Critical Path:

During the previous optimizations, the divider was kept disabled because it was originally implemented as a single-cycle unit. The divider was the most logic-intensive hardware block in the system, so keeping it disabled made it easier to first optimize the rest of the processor without causing system-wide timing failure.

After re-enabling the single-cycle divider, timing was no longer met. The WNS changed from a positive value to approximately **-85.989 ns**, with **420 failing endpoints**. This means the divider created a critical path that was far too long for the target clock period.

At the current period of **10.350 ns**, a negative slack of around **86 ns** indicates that the required period would need to be close to **100 ns** for the single-cycle divider to meet timing. This would reduce the design frequency to roughly **10 MHz**, which is a major decrease compared to the optimized CPU frequency.

The single-cycle divider also increased total on-chip power to approximately **0.153 W**, which is higher than expected for this project.

The critical path was located between the **EX1 pipeline register** and the **EX2 pipeline register**, where arithmetic operations such as the ALU, multiplier, and divider are performed.

Setup

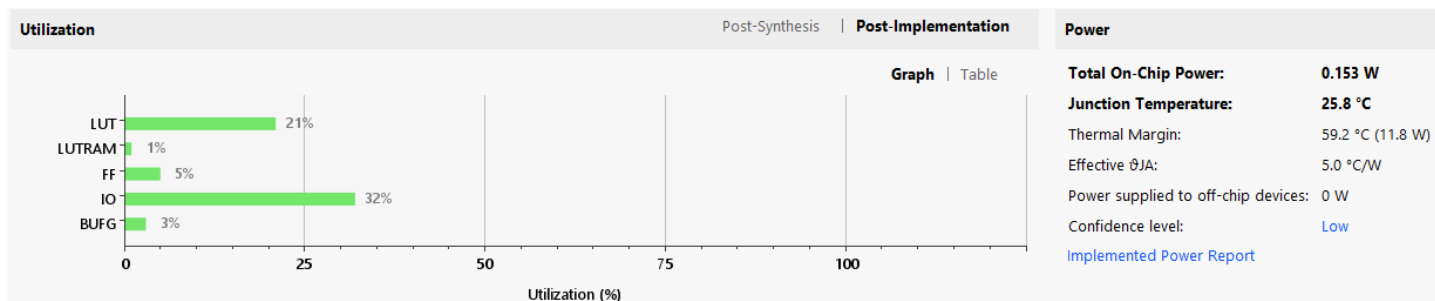
Worst Negative Slack (WNS): **-85.989 ns**
 Total Negative Slack (TNS): **-7053.964 ns**
 Number of Failing Endpoints: **420**
 Total Number of Endpoints: **3842**

Hold

Worst Hold Slack (WHS): **0.088 ns**
 Total Hold Slack (THS): **0.000 ns**
 Number of Failing Endpoints: **0**
 Total Number of Endpoints: **3842**

Timing constraints are not met.

Name	Slack ^{^1}	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
↳ Path 1	-85.989	330	40	35	ex1_pipe/data_out_1_EX_mux_EX2_reg[0]/C	ex2_pipe/HI_send_ME_reg[28]/D	96.296	62.171	34.125
↳ Path 2	-85.878	328	40	35	ex1_pipe/data_out_1_EX_mux_EX2_reg[0]/C	ex2_pipe/HI_send_ME_reg[24]/D	96.187	62.021	34.166
↳ Path 3	-85.865	329	40	35	ex1_pipe/data_out_1_EX_mux_EX2_reg[0]/C	ex2_pipe/HI_send_ME_reg[31]/D	96.170	61.881	34.289
↳ Path 4	-85.775	329	40	35	ex1_pipe/data_out_1_EX_mux_EX2_reg[0]/C	ex2_pipe/HI_send_ME_reg[30]/D	96.082	61.769	34.313
↳ Path 5	-85.735	330	41	35	ex1_pipe/data_out_1_EX_mux_EX2_reg[0]/C	ex2_pipe/HI_send_ME_reg[29]/D	96.041	61.707	34.334
↳ Path 6	-85.710	330	40	35	ex1_pipe/data_out_1_EX_mux_EX2_reg[0]/C	ex2_pipe/HI_send_ME_reg[27]/D	96.067	62.089	33.978
↳ Path 7	-85.708	330	40	35	ex1_pipe/data_out_1_EX_mux_EX2_reg[0]/C	ex2_pipe/HI_send_ME_reg[25]/D	96.015	62.063	33.952
↳ Path 8	-85.683	338	37	37	ex1_pipe/data_out_1_EX_mux_EX2_reg[0]/C	ex2_pipe/LO_send_ME_reg[28]/D	96.020	63.149	32.871
↳ Path 9	-85.677	339	37	37	ex1_pipe/data_out_1_EX_mux_EX2_reg[0]/C	ex2_pipe/LO_send_ME_reg[30]/D	95.995	63.281	32.714
↳ Path 10	-85.672	337	38	37	ex1_pipe/data_out_1_EX_mux_EX2_reg[0]/C	ex2_pipe/LO_send_ME_reg[17]/D	96.007	62.947	33.060



Root Cause Analysis:

Unlike multiplication, division does not map efficiently onto dedicated DSP blocks in the FPGA. A single-cycle divider requires a large amount of LUT-based combinational logic because the entire division operation must be completed in one clock cycle.

Division is naturally better suited for an iterative sequential implementation. When forced into a single cycle, the FPGA must create a large combinational divider using LUTs, which greatly increases logic delay, routing delay, LUT utilization, and power usage.

The utilization report shows that LUT usage increased to about **21%**, which corresponds to **4389 LUTs**. Before enabling the divider, LUT usage was closer to **10%**. This means a large portion of the FPGA logic was being used by the single-cycle divider, causing both timing and power issues.

Therefore, the root cause of the problem was that the divider was too large and too slow to complete in a single clock cycle.

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	4389	20800	21.10
LUTRAM	96	9600	1.00
FF	1883	41600	4.53
IO	34	106	32.08
BUFG	1	32	3.13

Optimization Strategy:

The best solution was to convert the divider into a multicycle unit. This reduces the amount of combinational logic required in one clock cycle and lowers LUT utilization.

Before Optimization:

The divider performed the full division operation in a single cycle using / and % operators. The quotient was stored in the lower 32-bit result, and the remainder was stored in the upper 32-bit result.

```
// If control unit signal is ALU_DIV (Division Signal):
if (ALU_Control_EX2 == ALU_DIV) begin

    // If the denominator isnt equal to zero then perform division:
    if (data_out_2_EX_mux_EX2 != 0) begin

        // Store the quotient in lower 32-bit register.
        div_LO_EX2 = $signed(data_out_1_EX_mux_EX2) / $signed(data_out_2_EX_mux_EX2);
        // Store the remainder in higher 32-bit register.
        div_HI_EX2 = $signed(data_out_1_EX_mux_EX2) % $signed(data_out_2_EX_mux_EX2);
    end
end
```

After Optimization:

```
always_ff @(posedge clk or posedge reset) begin

    // Reset.
    if (reset) begin
        cycle_div <= 0;
        div_operand_1 <= 0;
        div_operand_2 <= 0;
        remainder <= 0;
    end

    else if (Divide) begin

        // Start division.
        // If control unit signal is division and division cycle is zero, copy multiplexer values into a safe register.
        if (ALU_Control_EX2 == ALU_DIV && cycle_div == 0) begin

            div_operand_1 <= data_out_1_EX_mux_EX2;
            div_operand_2 <= data_out_2_EX_mux_EX2;
            remainder <= 0;
            cycle_div <= 1;
        end

        // Division iterations
        else if (cycle_div < 6'd33) begin

            // For 32 cycles, copy the operation results into safe registers.
            div_operand_1 <= next_div_operand_1;
            remainder <= next_remainder;
            cycle_div <= cycle_div + 1;
        end
    end
end
```

```

// Shift remainder and dividend left by 1 bit.
assign shifted_remainder = {remainder[30:0], div_operand_1[31]};
assign shifted_div_operand_1 = {div_operand_1[30:0], 1'b0};

always_comb begin

// Subtract remainder and divisor.
result = $signed(shifted_remainder) - $signed(div_operand_2);

// Default values for next cycle.
next_remainder = shifted_remainder;
next_div_operand_1 = shifted_div_operand_1;

// If the result of subtraction is negative, set the dividend bit index zero equal to 0.
if (result < 0) begin
    next_div_operand_1[0] = 1'b0;
end

// If the result of subtraction is positive, set the dividend bit index zero equal to 1 and set remainder equal to subtraction result.
else begin
    next_div_operand_1[0] = 1'b1;
    next_remainder = result;
end
end
end

```

The divider was redesigned as a multicycle restoring divider. Instead of completing the full division in one cycle, the divider processes one bit per cycle for 32 cycles.

The multicycle divider contains two main parts, sequential logic, and combinational logic to prepare the next state.

The sequential block stores the divider state. At the start of the division operation, the input operands are copied into internal registers so they are not lost in later pipeline cycles. During each division cycle, the updated quotient and remainder values are stored back into these registers.

The combinational block calculates the next division step. Each cycle, the dividend is shifted left by one bit. The remainder is also shifted left by one bit, with the remainder's lowest bit replaced by the highest bit of the dividend. Then the shifted remainder is subtracted from the divisor.

If the subtraction result is negative, the quotient bit is set to 0, and the shifted remainder is kept. If the subtraction result is non-negative, the quotient bit is set to 1, and the remainder is replaced with the subtraction result.

After 32 cycles, the final quotient and remainder are output. The quotient is stored in LO, and the remainder is stored in HI. At the same time, the multicycle control logic sends the required signals to write the results into the HI/LO registers.

FPGA Timing Results:

Setup

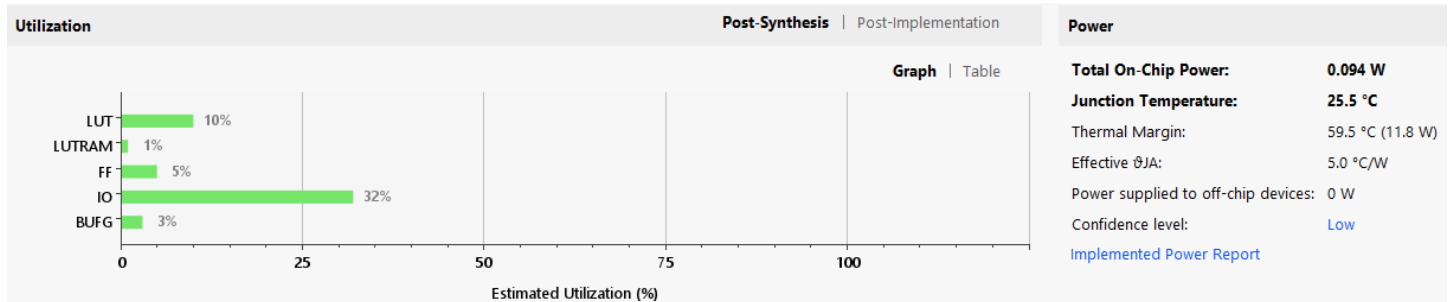
Worst Negative Slack (WNS): 0.092 ns
 Total Negative Slack (TNS): 0.000 ns
 Number of Failing Endpoints: 0
 Total Number of Endpoints: 4027

Hold

Worst Hold Slack (WHS): 0.113 ns
 Total Hold Slack (THS): 0.000 ns
 Number of Failing Endpoints: 0
 Total Number of Endpoints: 4027

All user specified timing constraints are met.

Name	Waveform	Period (ns)	Frequency (MHz)
sys_clk_pin	{0.000 5.000}	10.600	94.340



After implementing the multicycle divider, the design successfully met timing again, the final design maintained a frequency close to **94 MHz**, which is only slightly lower than the previous execute-stage optimized frequency.

The optimization also significantly reduced resource usage and power:

- LUT utilization decreased from about **21%** to **10%**
- Total LUT usage decreased by nearly **2200 LUTs**
- Total on-chip power decreased from **0.153 W** to **0.094 W**

This confirms that the multicycle divider greatly reduced the hardware cost of the division operation.

Tradeoffs:

The main tradeoff of the multicycle divider is latency. A division operation now takes **32 cycles** instead of one cycle.

This means dependent instructions, such as mflo or mfhi, must stall until the division result is ready. However, this latency tradeoff is acceptable because division is not usually as frequent as simple ALU operations, and the optimization greatly improves timing, power, and resource usage.

The multicycle divider also adds control complexity because the design now requires cycle counting, quotient/remainder update logic, stall handling and HI/LO write control.

However, this added complexity is worth it because the divider no longer breaks timing closure.

Conclusion:

The single-cycle divider was a major timing bottleneck in the processor. It created a very large critical path, caused timing failure, increased LUT utilization, and raised total power consumption.

After converting the divider into a multicycle restoring divider, the processor successfully met timing at **94.340 MHz** with positive WNS. LUT utilization dropped about **11%**, and total on-chip power decreased by **0.059 W**.

The optimization successfully traded division latency for improved timing closure, lower area usage, and lower power consumption. The divider no longer dominates the processor's critical path.